

GENERALIZED ADAPTIVE MARGIN SOFTMAX

GAM-Softmax

Can we make the softmax margin *smart*
instead of a single number?

Ashish Gupta

One RTX 3050 laptop GPU · 23 May – 31 July 2026

Where this talk goes

1. **Introduction** — what softmax is, and the flaw in how we train it
2. **Background** — AS-Softmax and its single number δ
3. **Methodology** — turning δ into a function
4. **Experimentation** — what I ran and what I measured
5. **Results** — the numbers, and the verdict

The whole talk in one line

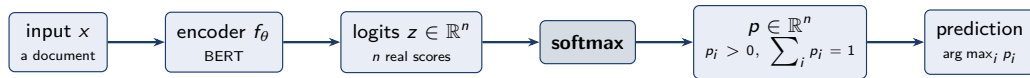
A classifier is normally told “*be 100 % sure, forever*”. A better instruction is “*win by a margin δ , then stop*”. I asked whether that margin should be **one number** or **a function**.

SECTION 1 OF 5

Introduction

Where the softmax sits

The last layer of essentially every classifier



The job it performs

The network emits n unbounded real numbers (“logits”) — they can be negative, they do not sum to anything. Softmax is the map

$$\mathbb{R}^n \longrightarrow \Delta^{n-1}$$

that turns them into a *probability distribution* over the n classes.

Where it is used

- every classifier: text, image, audio
- every language model: the distribution over the next token ($n \approx 50,000$)
- attention weights inside a Transformer
- action probabilities in reinforcement learning

The softmax function

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad i = 1, \dots, n$$

Why the formula is built this way

1. **Positive:** $e^{z_i} > 0$ always $\Rightarrow p_i > 0$.
2. **Normalised:** dividing by the sum forces $\sum_i p_i = 1$.
3. **Order-preserving:** e^x is strictly increasing, so $z_i > z_j \Leftrightarrow p_i > p_j$. The prediction is unchanged.
4. **Smooth:** differentiable everywhere $\Rightarrow$ trainable by gradient descent.

Two properties I will use later

- **Shift invariance:** $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$, since $e^{z_i+c} = e^c e^{z_i}$ cancels top and bottom. *Only differences of logits carry information* — which is why a margin is defined on a *difference*.
- **Temperature:** $\text{softmax}(z/T)$ tends to $\arg \max$ as $T \rightarrow 0$ and to uniform as $T \rightarrow \infty$. Softmax is the *smooth* version of $\max$ — hence the name.

A worked example

Four classes, one document

exponentiate $\longrightarrow$ sum $\longrightarrow$ divide

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

class	z_i	e^{z_i}	p_i
sport	3.2	24.53	0.775
politics	1.3	3.67	0.116
medicine	0.2	1.22	0.039
religion	0.8	2.23	0.070
total		31.65	1.000

Check the shift invariance. Subtract 3.2 from every logit:

$$z' = (0, -1.9, -3.0, -2.4)$$

$$e^{z'} = (1, 0.150, 0.050, 0.091), \quad \text{sum} = 1.291$$

$$p' = (0.775, 0.116, 0.039, 0.070) = p$$

Identical. The absolute size of a logit means nothing; only the gaps between logits do. **This is the whole reason a “margin” is the natural thing to control.**

How we train it: cross-entropy

And the flaw hiding in its gradient

With true class t and one-hot label y , the loss is

$$L = - \sum_{i=1}^n y_i \log p_i = -\log p_t \quad (\text{only the true-class term survives})$$

Differentiating through the softmax gives a famously clean gradient:

$$\frac{\partial L}{\partial z_j} = p_j - y_j \quad \implies \quad \frac{\partial L}{\partial z_t} = p_t - 1, \quad \frac{\partial L}{\partial z_j} = p_j \quad (j \neq t)$$

Read the gradient.

- It vanishes *only* when $p_t = 1$ exactly and every $p_j = 0$.
- But $p_i = e^{z_i} / \sum_j e^{z_j} > 0$ strictly — $p_t = 1$ is **unreachable** for finite logits.

Cross-entropy never stops pushing. At $p_t = 0.99$ it still pulls with -0.01 , and it will keep pulling for as long as you train. There is no notion of “this example is done”.

Why “never stops” is a real problem

1. **Wasted effort.** With $n = 20$ classes, 19 of the 20 slots per example get gradient at every step — including classes that were hopeless from epoch one.
2. **Overconfidence.** The objective's only fixed point is total certainty, so probabilities get pushed to extremes and stop being honest confidences.
3. **Memorisation of wrong labels.** If a training label is *incorrect*, cross-entropy pushes the model to fit it just as hard. Given enough steps it succeeds.

The point that drives this project

A deep network has enough capacity to fit *arbitrary* labels, including random ones. Cross-entropy gives it no reason to stop — so with noisy labels, it memorises the noise.

The natural fix: **stop pushing once the answer is clearly right**. Now we need to define “clearly”.

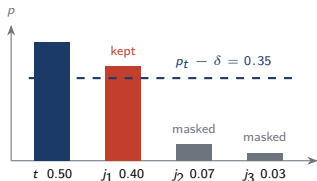
SECTION 2 OF 5

Background

AS-Softmax (Lv et al., 2023) — the method I build on

The rule. For a training example with true class t , *drop* a competing class j from the loss as soon as it is beaten by a margin δ :

$$\text{mask } j \iff p_t - p_j \geq \delta, \quad \mathcal{K}(x) = \{t\} \cup \{j \neq t : p_t - p_j < \delta\}, \quad L_{\text{AS}} = -\log \frac{e^{z_t}}{\sum_{k \in \mathcal{K}(x)} e^{z_k}}$$



With $\delta = 0.15$: j_1 is only 0.10 behind, so it is *still a threat* and stays in the loss. j_2, j_3 are 0.43 and 0.47 behind — beaten by more than δ , so they are dropped and contribute no gradient.

The instruction changes from “*beat everything by everything*” to “*beat everything by δ , then stop*”. Reported: better accuracy, better calibration, $\approx 1.2\times$ faster.

Three details a referee asks about: the test is on *probabilities*, not logits; the mask is *detached*; it is recomputed *every step*, never cached.

The gap in AS-Softmax — and my hypothesis

δ is a single scalar

The same number for “sport vs. politics” and “sport vs. hardware”. The same number for a clean example and a mislabelled one. The same number at step 1 and at step 10,000.

$$\delta \longrightarrow \delta_{t,j}(x, \tau) \quad \tau = \frac{\text{current step}}{\text{total steps}} \in [0, 1]$$

axis	why it might need its own margin
class-pair (t, j)	confusable pairs need a bigger gap than obvious ones
sample x	an ambiguous or mislabelled example is not like a clean one
time τ	early training is noisy; start loose, tighten later

	hypothesis
H0	AS-Softmax beats plain cross-entropy (assumed, never tested)
H1	class-pair structure helps
H2	per-sample structure helps
H3	the axes compose

Success criterion, fixed before running anything: $\geq +0.3\%$ accuracy over AS-Softmax, averaged over seeds. Writing it down in advance is what stops you moving the goalposts afterwards.

SECTION 3 OF 5

Methodology

GAM-Softmax: the general form

$$\mathcal{K}(x) = \{t\} \cup \{j \neq t : p_t - p_j < \delta_{t,j}(x, \tau)\}, \quad L = -\log \frac{e^{z_t}}{\sum_{k \in \mathcal{K}(x)} e^{z_k}}$$

Same loss as before — the *only* change is that the threshold is now a function rather than a constant.

Reduction property (unit-tested)

If $\delta_{t,j}(x, \tau) \equiv \delta$ is constant, then $\mathcal{K}$ is *exactly* AS-Softmax's kept set, so $L \equiv L_{AS}$ — not approximately, identically.

Two familiar losses fall out

$\delta \geq 1 \Rightarrow$ nothing is ever masked (since $p_t - p_j \leq 1$)
 $\Rightarrow$ **plain cross-entropy**.
 δ constant $\Rightarrow$ **AS-Softmax**.

This matters for honesty, not elegance: because the general form *contains* the baseline, any difference in the results is attributable to the margin function alone — not to a different loss, a different optimiser, or a different bug.

Enforced by a test that asserts equality of the two losses to floating-point tolerance. 70 unit tests, ≈ 3 s.

M3 — the class-pair axis

$$\delta_{t,j} = \delta_{\min} + (\delta_{\max}(\tau) - \delta_{\min}) \cdot \sigma\left(\frac{u_t \cdot v_j}{\sqrt{k}}\right), \quad u, v \in \mathbb{R}^{n \times k}, \quad k = 8$$

Reading the formula

- every class c gets two learnable vectors u_c, v_c of length $k = 8$; their inner product scores the *ordered* pair (t, j)
- σ (the logistic sigmoid) squashes that score into $(0, 1)$, so $\delta_{t,j}$ can never leave $[\delta_{\min}, \delta_{\max}]$
- $\delta_{\max}(\tau)$ is the *time* axis: a warm-up schedule
- dividing by $\sqrt{k}$ keeps the pre-sigmoid scale $O(1)$ — the same variance argument as in attention

Why low-rank, not a full table?

A free margin per ordered pair is $n(n-1) = 380$ unconstrained numbers for $n = 20$, with no sharing between classes.

The low-rank form uses $2nk = 320$ and *forces* classes to share structure — which is the actual hypothesis: that pair difficulty is low-dimensional.

M4 — the sample axis

Step 1 — how suspicious is this example, relative to its batch?

$$s_i = \sigma\left(\frac{\bar{p}_t - p_{t,i}}{\text{std}(p_t) \cdot \text{temp}}\right) \in (0, 1), \quad s_i > 0.5 \iff \text{less confident than its peers}$$

Step 2 — move the margin by that amount

$$\delta_i(\tau) = \text{clamp}\left(\underbrace{\delta_{\text{base}}(\tau)}_{\text{time axis}} - \beta(\tau)(2s_i - 1), \delta_{\text{floor}}, \delta_{\text{ceil}}\right)$$

- suspicious ($s_i \rightarrow 1$) $\Rightarrow \delta$ down $\Rightarrow$ more classes masked $\Rightarrow$ less gradient
- confident ($s_i \rightarrow 0$) $\Rightarrow \delta$ up, but capped at AS-Softmax's own δ — the axis is **one-sided**, so any difference comes purely from what we do to suspicious samples

Two deliberate design choices

$(2s_i - 1)$ averages to ≈ 0 over a batch, so this varies δ across samples without changing the overall masking level — that stays the time axis's job, keeping the axes separable.

$\beta(\tau) = 0$ for the first 30 % of training: early on p_t is near-random and carries no signal about which labels are wrong.

The idea worth keeping: let δ go negative

Nothing in the mathematics requires a margin to be positive. Since $p_t - p_j \in [-1, 1]$, the value $\delta = -0.85$ is a perfectly legal threshold. It reads:

“drop class j even if it is currently beating the training label by up to 0.85.”

On a mislabelled example

- the model keeps p_t low (the label is wrong)
- so with $\delta > 0$, *no* competitor ever clears the bar
- nothing is masked $\Rightarrow$ full cross-entropy $\Rightarrow$ pushed to fit the wrong label forever
- **that is exactly how a network memorises noise**

With $\delta < 0$, the algebra ends it

$$\mathcal{K}(x) = \{t\} \Rightarrow p_t^{\text{renorm}} = \frac{e^{z_t}}{e^{z_t}} = 1$$

$$\Rightarrow L = -\log 1 = 0 \Rightarrow \nabla L = 0$$

The suspicious sample *removes itself* from training
— no threshold rule, no sample-selection heuristic.

The reframing

The noisy-label literature’s “small-loss trick” — drop the high-loss examples — is exactly the special case $\delta < 0$ of a *margin*. No second network, no auxiliary loss, no selection schedule: one line of configuration.

SECTION 4 OF 5

Experimentation

The setup

Constraint that shaped everything

One **RTX 3050 laptop, 6 GB** (the plan assumed a 3090/4090). One 20-Newsgroups run ≈ 25 min $\Rightarrow$ a properly seeded sweep is a day of wall clock.

Testbed

- 20 Newsgroups, 20 classes, headers/footers/quotes *stripped* — otherwise BERT reaches $\approx 99\%$ from metadata and no method can separate from another
- BERT-base, 4 epochs, batch 16, lr 2×10^{-5} , seq len 128
- 6 configs, **identical** except the loss: block; 2 seeds (42, 43)

Label noise — and why it is a *paired* comparison

40 % symmetric label noise on the training set; validation labels stay clean.

The noise seed is fixed and *decoupled* from the run seed, so every method sees the *identical* corrupted labels. Methods are compared on the same corrupted dataset, not on statistically similar ones.

One config file = one experiment = one row of the results table. 9 loss functions, 70 unit tests running in ≈ 3 s. The null result is not a code failure.

What I measure — and the arithmetic behind it

Accuracy alone had been flat in every earlier experiment, so E1 measures the *mechanism* directly.

- **peak** and **final** validation accuracy, and the **decay** between them — the overfitting trajectory
- **memorisation rate:** of the training examples whose labels were *corrupted*, the fraction the model now predicts *as their corrupted label*
- **recovery rate:** same examples, fraction predicted as their *true* label despite being trained on a wrong one
- **masked ratio**, and 15-bin **ECE** (calibration)

The no-memorisation floor

With 40 % noise, where a flip never returns the original label, a model that learned the *true* function and memorised *nothing* scores exactly

$$0.6 \times 100 \% + 0.4 \times 0 \% = 60.0 \%$$

on the labels it was given.

Everything above 60 is memorised noise — a quantity, not an impression.

Every experiment, in order

#	testbed	question	result
1	SST-5, clean, 3 seeds	baselines sane?	softmax 51.1 %, AS-Softmax 52.2 % $\Rightarrow$ AS $>$ CE by +1.1 $\checkmark$
2	SST-5, clean, 3 seeds	H1, fixed margin	M3 52.10 vs AS 52.17 = -0.06
3	SST-5, clean, 3 seeds	H1, learnable margin	M3' 51.83 vs AS 52.17 = -0.33
4	SST-5, tuning screen	is H1 under-tuned?	higher margin LR hurt <i>monotonically</i> (52.5 $\rightarrow$ 50.6 $\rightarrow$ 50.1) $\Rightarrow$ not a tuning artefact
5	20NG, clean, 1 seed	testbed with head-room?	GAM 71.18 $>$ AS 70.59, but plain CE 71.46 wins $\Rightarrow$ AS $<$ CE
6	SST-5, noise {0, .2, .4}	does a hard regime rescue it?	all gaps inside $\pm 2\%$ seed scatter $\Rightarrow$ inconclusive
7	20NG, 40 % noise, 2 seeds	the decisive test	next section

H0 kept failing. AS-Softmax beat cross-entropy on SST-5 (+1.1), lost on 20NG (-0.9), and was a wash under noise. Three of four setups gave the method I extend *no edge* over the simplest possible baseline.

The one repeatable GAM signal was stability, not accuracy: lowest seed-to-seed variance at every noise level, and still *improving* at the last epoch where the baselines had peaked and turned over.

SECTION 5 OF 5

Results and Demonstration of Final Result

E1 — the decisive experiment

20 Newsgroups, 40 % symmetric label noise, BERT-base, 2 seeds (42, 43)

method	peak val %	final val %	decay	memorised noise (pts over floor)	mem @final %	recovers true %
softmax (plain CE)	67.59 ± 0.71	64.19 ± 0.92	-3.40	+11.5	33.7 ± 0.4	41.2
as_softmax (scalar δ)	67.56 ± 0.34	64.72 ± 0.20	-2.84	+13.6	39.9 ± 1.8	44.1
gam_m3 (class-pair)	67.68 ± 0.81	65.21 ± 1.42	-2.47	+11.4	35.2 ± 1.7	45.9
gam_m4 (sample)	68.09 ± 0.05	66.96 ± 0.42	-1.13	+3.1	19.3 ± 0.7	57.3

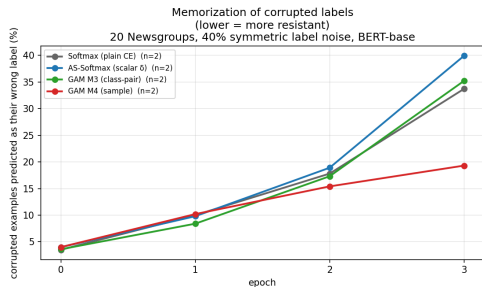
Read the accuracy column first — it is flat

Peak accuracy spans 67.56 to 68.09: half a point across all four methods. Plain cross-entropy alone spans **1.0 point** between its two seeds. **The accuracy column contains no result.**

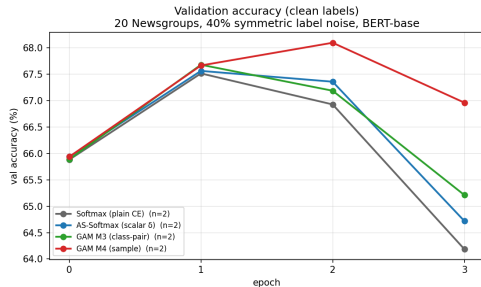
Now read the memorisation column

The same two seeds of plain cross-entropy span only **0.6 points** of memorisation. So a **14-point** gap is $\approx 20\times$ the measurement's own noise — it cannot be a lucky seed.

The output, in two pictures



Lower is better. All methods rise — M4's curve flattens while the rest accelerate.



Everything peaks at epoch 1 and decays. M4 decays least — -1.13 points against cross-entropy's -3.40.

Same runs, two views. Left: by the last epoch, AS-Softmax has memorised *more* corrupted labels than plain cross-entropy (39.9 % vs 33.7 %) — the opposite of what the margin-loss story predicts. Right: the four curves are almost on top of each other until the overfitting phase separates them.

What the numbers actually say

1. The precondition failed.

AS – CE = -0.03 points over two seeds (-0.29 and $+0.23$: a coin flip). *Fourth testbed in a row.* The method I set out to generalise has, in my hands, no advantage over plain cross-entropy to generalise.

2. AS-Softmax memorises *more* than plain CE.

$+13.6$ vs $+11.5$ points over the floor, on *both* seeds ($+7.8$, $+4.6$). This contradicts the standard story — and my own reason for moving to noisy labels.

Mechanism: masking retires the *easy* negatives first — what a correctly-labelled example runs out of early. So an ever-larger share of the gradient goes to the still-confusable examples, which are exactly the mislabelled ones. **Scalar masking concentrates effort onto the noise.**

3. The sample axis does what it was designed to do.

(2-seed means)	CE	AS	M3	M4
memorised noise (pts)	11.5	13.6	11.4	3.1
recovers true label	41.2	44.1	45.9	57.3
peak-to-final decay	3.40	2.84	2.47	1.13
non-target slots masked	0 %	56 %	49 %	95 %

Four independent measurements moving together in the predicted direction, driven by a mechanism I can point at in the code: δ goes negative $\rightarrow$ the suspect sample's competitors are all masked $\rightarrow$ its loss contribution vanishes.

Verdict — stated as it is

	hypothesis	verdict
H0	AS > plain CE	NOT SUPPORTED — 3 of 4 testbeds show no advantage
H1	class-pair helps	NOT SUPPORTED — +0.12 over AS, inside a 1.0-pt noise band
H2	sample axis helps	SPLIT — peak accuracy not established; memorisation cut $\approx 4\times$
H3	the axes compose	UNTESTED — implemented, run cut for compute

finding	confidence
M4 cuts memorisation $\approx 4\times$ (-14.4 pts)	Confirmed (both seeds; $\approx 20\times$ the metric's spread)
AS-Softmax memorises <i>more</i> than CE	Confirmed (both seeds)
M4 gains at the final epoch (+2.77)	Confirmed (direction) (magnitude varies $2\times$)
H0 and H1 null	Confirmed (4 testbeds)
M4 gains at <i>peak</i> accuracy	Not established (one seed)
M4 is more seed-stable	Suggestive only ($n = 2$)
Calibration	No claim

The honest limit

M4's mean peak is +0.51 over CE — which would clear my +0.3 criterion. **I do not claim it.** Per seed it is -0.03 and $+1.04$: the entire mean comes from one seed where cross-entropy had a bad run. What *does* replicate is the *final-epoch* gain: +3.72 and +1.83, both positive, mean +2.77.

Conclusion

In one sentence

Generalising AS-Softmax's scalar margin along the **class-pair** axis produced no measurable benefit on any of four testbeds; generalising it along the **sample** axis — by letting the margin go *negative*, which makes small-loss sample rejection a special case of a margin — did not reliably improve peak accuracy either, but **cut memorisation of corrupted labels roughly fourfold** and left the final-epoch model **2.8 points** better than cross-entropy under 40 % label noise.

What I would claim in writing

- the class-pair negative result, replicated over four testbeds and shown not to be a tuning artefact
- that AS-Softmax *increases* memorisation under label noise, contradicting the standard rationale for margin losses in that regime — measured directly, not inferred from accuracy
- the negative-margin reframing (small-loss rejection *is* a margin), with evidence that it works mechanically

What I would *not* claim

- that GAM-Softmax improves accuracy. It does not.
- any state-of-the-art noisy-label claim — I never ran the real baselines (co-teaching, DivideMix, GCE). M4 beats *cross-entropy and AS-Softmax at the final epoch*; that is a much weaker statement, and it is the only one the data supports.
- any calibration claim (the ECE gap is *underconfidence*; largely fixable by temperature scaling, which I did not run)

Limits, and what a follow-up should do differently

What would have to be true for this to be wrong

- 2 seeds, 1 dataset, 1 noise level, 1 architecture, 4 epochs. Two seeds is enough to kill a hypothesis and not enough to establish a small effect — which is exactly why the memorisation gap is claimed and the accuracy gap is not.
- β was never swept — the cleanest evidence for a mechanism is that the effect scales with its knob, and that is the run I could not afford
- symmetric noise is the easy case; real noise is instance-dependent, where “the model disagrees with this label” is a much weaker signal of corruption
- weak baselines by design — CE and AS-Softmax, not methods built for noisy labels

What a follow-up should do differently

1. **validate H0 in week 1**, not week 8. One week reproducing the baseline’s published gain would have redirected the whole project.
2. pick a regime where masking is *forced* to matter — 10k+ classes, or genuinely long-tailed data. Balanced text classification with a strong pretrained encoder is close to the worst case: plain CE is very hard to beat there.
3. instrument the mechanism from day 1. I read only accuracy for six weeks; it was flat everywhere and told me nothing about *why*.
4. keep the negative-margin framing, and test it against the real noisy-label baselines. It is one config line away.

BACKUP SLIDES

For questions

Backup — head-to-head, reported per seed

A two-seed mean can hide the fact that one seed did all the work, so here is every comparison split out.

vs. plain CE	peak (s42 / s43)	final (s42 / s43)	memorisation (s42 / s43)
as_softmax — CE	−0.29 / +0.23	+1.33 / −0.26	+7.8 / +4.6
gam_m3 — CE	+0.16 / +0.01	+0.67 / +1.38	+0.6 / +2.4
gam_m4 — CE	−0.03 / + 1.04	+3.72 / +1.83	−14.6 / −14.2

Read the M4 row across: the peak column disagrees with itself, the final and memorisation columns agree on both seeds. That difference is the entire basis for which claims I make and which I refuse to.

Backup — the calibration caveat

The margin variants have far *worse* ECE: 0.42–0.47 against plain cross-entropy's 0.19. Read naively that is a defeat. It is not what is happening.

- M4's mean predicted confidence is ≈ 0.25 while its accuracy is ≈ 0.67
- so the model is systematically **under**confident, not overconfident
- by construction: the margin objective *stops pushing* p_t upward once the gap is met, so the probabilities stay compressed

Underconfidence is a benign failure mode compared with cross-entropy's direction, and it is largely removable by temperature scaling — which I did not run. So: **no calibration claim in either direction.**

Backup — reproducing all of it

```
conda activate gam
```

```
python -m pytest tests/ -v                - 70 tests, ~3 s
python experiments/final_experiment.py      - E1, resumable (~6 h)
python experiments/final_experiment.py -only-block 1
```

- FINAL_REPORT.md — the full write-up
- HOW_WE_VARY_DELTA.md — the method in plain language
- runs/final_20ng/ — per-run JSON with full epoch histories, plus summary.md
- 9 losses in gam_softmax/losses/
- margins in gam_softmax/margins/
- one config = one experiment = one table row
- run 2026-07-28/31, commit b2ff750